Kiln Corner Thermal Simulator – Software Implementation Plan

1. Overview & System Requirements

This document specifies the software architecture, build system, data structures, and configuration schema for the **Rohde Ecotop 60 Corner Thermal & Airflow Simulator** (`kiln_thermal_sim`) on Debian Linux.

The implementation is designed around five core principles:

1. **Debian-Native Security (`apt` Only, Zero `pip`):** All runtime, scientific, and serialization dependencies are installed exclusively from official Debian distribution packages to avoid PyPI supply-chain risks.

2. **Immutable Experiment Configs & Cryptographic Provenance:** Input configurations are stored in human-readable Google Protocol Buffer text format (`.textproto`). The simulator treats input files as read-only, computes their raw-byte `md5sum`, and embeds both the hexadecimal MD5 digest and a full snapshot of the input configuration inside a timestamped output `.out.textproto` file.

3. **Single-Run Deterministic Scope:** The core physics simulator executes a single deterministic experiment per invocation with zero side effects.

4. **Vectorized Solver with Ergonomic Inspection:** Internally, the physics engine operates on contiguous NumPy arrays (Structure-of-Arrays) in strict SI base units (m, K, W, m^3/s), using a vectorized diagonal-Newton solver with exact analytical derivatives and pre-factored radiosity LU decomposition. High-level single-patch lookup methods (`SurfaceMesh.get_patch`, `get_patches_by_type`, `find_patch_at_coord`) return self-contained `PatchView` records for inspection and debugging.

5. **Decoupled Multi-Core Parametric Sweeps:** Large-scale parameter exploration is implemented via a standalone driver (`run_sweep.py`) that utilizes multi-process parallelism (`concurrent.futures.ProcessPoolExecutor`), caches geometry and view factor systems across configurations sharing identical spatial meshes (`geometry_cache_key`), supports fast testing via reservoir sampling (`--quick`), and generates consolidated CSV tables (`sweep_results.csv`).

---

2. Debian Package Dependencies & Build Tooling

All required libraries and tools are installed via `apt`:

```
sudo apt update && sudo apt install -y \
    python3 \
    python3-absl \
    python3-numpy \
    python3-scipy \
    python3-protobuf \
    protobuf-compiler \
    python3-pytest \
    bazel \
    elpa-protobuf-mode
```

- **`python3-absl`**: Google Abseil Python application framework (`absl.app`, `absl.flags`, `absl.logging`, and `absl.testing.absltest`).

- **`python3-numpy`**: Vectorized array storage, coordinate geometry, and dense linear systems (`scipy.linalg.lu_factor`, `lu_solve` for the radiosity matrix).

- **`python3-scipy`**: 1D root-finding (`scipy.optimize.root_scalar` with Brent's method) for floor and ceiling point sensor thermal equilibria.

- **`python3-protobuf` & `protobuf-compiler`**: Compiles `.proto` schemas and parses/serializes human-readable `.textproto` files via `google.protobuf.text_format`.

- `bazel`: Manages the hermetic build graph, `proto_library` / `py_proto_library` targets, `py_library` modules, `py_binary` entry points, and `py_test` suites.

To ensure Bazel uses the system Python 3 interpreter and resolves Debian's `/usr/lib/python3/dist-packages`, the workspace root includes `.bazelrc`:

```
build --action_env=PATH="/usr/bin:/bin"
build --action_env=PYTHONPATH="/usr/lib/python3/dist-packages"
test --test_env=PYTHONPATH="/usr/lib/python3/dist-packages"
```

---

3. Coding Standards & Style Rules (Mandated in `README.md`)

All code in this repository adheres to the following stylistic and architectural rules:

1. **Google Python Style Guide:** All Python source and test files follow the Google Python Style Guide, including 2-space indentation, `CapWords` class names, `lower_with_under` function and variable names, module-level docstrings, `Args:` / `Returns:` / `Raises:` docstring blocks, and strict PEP 484 type annotations.

2. **Protocol Buffers Style Guide:** All `.proto` definitions follow the Google Protobuf Style Guide, using `CamelCase` message and enum names, `lower_snake_case` field names, and `CAPITALS_WITH_UNDERSCORES` enum values prefixed with the enum type name. All input submessages are nested inside `SimulationConfig`, and all output submessages are nested inside `SimulationOutput`.

3. **Abseil Application Lifecycle:** All executable entry points use `absl.app.run()`, command-line arguments are parsed via `absl.flags`, and diagnostic output uses `absl.logging`. Unit tests inherit from `unittest.TestCase` / `absl.testing.absltest`.

4. **Structured Per-Run Logging:** The logging setup (`io_manager.configure_absl_logging`) automatically creates a unique timestamped subdirectory `<yyyymmdd-hhmmss.s>_<binary_name>` under `/usr/local/google/tmp` (or `FLAGS.log_dir`) to prevent log file collisions during parallel test and sweep runs.

5. **Explicit Enum Value Assignment:** Every `enum` in both `.proto` files and Python `enum.IntEnum` classes explicitly assigns integer values (never using `enum.auto()`), starting with an explicit `UNSPECIFIED = 0`.

6. **Top-Level Message Encapsulation:** All input submessages are encapsulated inside `SimulationConfig`, and all output submessages are encapsulated inside `SimulationOutput`.

---

4. Repository & Bazel Workspace Layout

```
kilnsim/
+-- MODULE.bazel                  # Bazel Bzlmod workspace configuration
+-- MODULE.bazel.lock             # Bazel module dependency resolution lockfile
+-- BUILD.bazel                   # Root build file (run_sweep binary and configs filegroup)
+-- README.md                     # System requirements, style rules, and Bazel usage instructions
+-- proto/
|   +-- BUILD.bazel               # proto_library and py_proto_library build targets
|   +-- kiln_sim.proto            # Hierarchical SimulationConfig and SimulationOutput schema
+-- configs/
|   +-- baseline_205mm.textproto  # Annotated baseline default experiment configuration
|   +-- ...                       # Additional design exploration textproto configurations
+-- results/                      # Default output directory for timestamped .out.textproto files
+-- src/
|   +-- BUILD.bazel               # py_library and py_binary targets for physics modules
|   +-- __init__.py               # Top-level kiln_sim_lib package export
|   +-- main.py                   # Abseil CLI entry point for single simulation execution
```

```
|   +-- io_manager.py             # MD5 hashing, textproto serialization, and Abseil logging setup
|   +-- params.py                 # Frozen SI-unit SimParams dataclass, validation, derived datums
|   +-- mesh.py                   # SurfaceMesh SoA storage + PatchView single-patch query API
|   +-- view_factors.py           # Diffuse + specular reflector view factors and sensor vectors
|   +-- chimney.py                # 1D buoyant channel flow, friction, solid contact, AAC conduction
|   +-- plume_room.py             # Corner-modified MTT plume, hood capture, 2-zone room model
|   +-- solver.py                 # Vectorized diagonal-Newton coupled thermal-fluid solver
|   +-- metrics.py                # Engineering safety metrics, floor/ceiling sensors, output builder
+-- run_sweep.py                  # Standalone multi-core parametric sweep generator and CSV exporter
+-- sweep_configs/                # Generated .textproto files from parametric sweep combinations
+-- sweep_results/                # Solved .out.textproto files from parametric sweep execution
+-- sweep_results.csv             # Consolidated CSV table of swept parameters and safety metrics
+-- tests/
    +-- BUILD.bazel               # py_test targets for unit and integration tests
    +-- params_test.py            # Verifies vertical datum stack, derived datums, and bounds checks
    +-- mesh_test.py              # Verifies PatchView lookup by ID, 3D coordinate, and SurfaceType
    +-- view_factors_test.py      # Verifies view factor reciprocity, disk formulas, and ray blocking
    +-- chimney_test.py           # Verifies 1D buoyant draft, friction, flush wall, and cavity limits
    +-- solver_test.py            # Verifies global energy conservation and Newton convergence
    +-- run_sweep_test.py         # Verifies sweep point enumeration, logging, and reservoir sampling
```

---

5. Protocol Buffer Schema (`proto/kiln_sim.proto`)

```
// Protocol Buffer schema for the Rohde Ecotop 60 corner thermal and airflow
// simulation model.

syntax = "proto3";

package kiln_sim;

// Identifies the physical surface category of a discretized mesh patch.
enum SurfaceType {
  SURFACE_TYPE_UNSPECIFIED = 0;
  SURFACE_TYPE_KILN_SIDE = 1;
  SURFACE_TYPE_KILN_LID = 2;
  SURFACE_TYPE_KILN_BOTTOM = 3;
  SURFACE_TYPE_SHIELD_LEFT = 4;
  SURFACE_TYPE_SHIELD_BACK = 5;
  SURFACE_TYPE_GYPSUM_LEFT = 6;
  SURFACE_TYPE_GYPSUM_BACK = 7;
  SURFACE_TYPE_REFLECTOR_TOP = 8;
  SURFACE_TYPE_SSR_BOX = 9;
}

// Top-level immutable input configuration message for a single simulation run.
message SimulationConfig {
  // Dimensions and clearances of the calcium silicate wall heat shields.
  message ShieldGeometry {
    // Horizontal width of the left-wall heat shield along the +x axis (mm).
    double w_shield_left_mm = 1;
    // Horizontal width of the back-wall heat shield along the +y axis (mm).
    double w_shield_back_mm = 2;
    // Vertical inlet air gap between the floor (z = 0) and shield bottom edge (mm).
    double g_floor_mm = 3;
    // Vertical outlet air gap between the shield top edge and ceiling (mm).
    double g_ceil_mm = 4;
```

```
  // Thickness of the calcium silicate shield board (mm).
  double t_shield_mm = 5;
}

// Geometry of the ventilated air channel behind the wall heat shields.
message ChimneyGeometry {
  // Depth of the air gap between the AAC block wall and the shield back face (mm).
  double d_chimney_mm = 1;
  // Fraction of the behind-shield air gap cross-section obstructed by battens (0.0 to 1.0).
  double chimney_blockage_fraction = 2;
  // Whether the vertical outer side edges of the chimney gap are closed off.
  bool seal_chimney_sides = 3;
}

// Dimensions of the structural walls and room enclosure.
message RoomGeometry {
  // Total interior wall height from structural screed (z = 0) to ceiling (mm).
  double h_wall_mm = 1;
  // Thickness of standard Class A2 gypsum plasterboard outside the shield zone (mm).
  double t_gypsum_mm = 2;
  // Thickness of the structural aerated autoclaved concrete (AAC) block wall (mm).
  double t_aac_mm = 3;
  // Interior room length along the +x axis from back wall to front wall (mm).
  double l_room_mm = 4;
  // Interior room width along the +y axis from left wall to partition wall (mm).
  double w_room_mm = 5;
  // Total width of the glass sliding door assembly on the front wall (mm).
  double w_glass_door_mm = 6;
  // Horizontal open slot width of the ajar sliding door during firing (mm).
  double w_door_open_mm = 7;
  // Vertical height of the sliding door opening (mm).
  double h_door_mm = 8;
  // Behind-shield air channel geometry associated with the corner walls.
  ChimneyGeometry chimney = 9;
}

// Geometry and placement of the cylindrical kiln drum and lid.
message KilnGeometry {
  // Clear horizontal standoff from the left shield front face to the kiln jacket (mm).
  double s_kiln_left_mm = 1;
  // Clear horizontal standoff from the back shield front face to the kiln jacket (mm).
  double s_kiln_back_mm = 2;
  // Outer diameter of the stainless steel kiln cylinder jacket (mm).
  double d_kiln_mm = 3;
  // Height of the cylindrical kiln drum body excluding the lid (mm).
  double h_drum_mm = 4;
  // Thickness of the closed refractory kiln lid (mm).
  double t_lid_mm = 5;
  // Clear vertical distance from the floor pad top surface (z_pad_top) to the kiln bottom plate (mm).
  double h_kiln_base_mm = 6;
}

// Position and capture state of the side-mounted exhaust dilution bung.
message BungGeometry {
  // Azimuthal angle of the exhaust bung counter-clockwise from +x (degrees).
  double bung_angle_deg = 1;
  // Radial distance from the vertical kiln centerline axis to the dilution cup (mm).
```

```
  double r_bung_mm = 2;
  // Vertical offset from the kiln bottom plate to the open top rim of the cup (mm).
  double z_bung_offset_mm = 3;
  // True if exhaust bung gas is captured directly via a local funnel/hose.
  bool bung_captured = 4;
}

// Dimensions and mounting orientation of the louvered SSR electrical enclosure.
message SsrGeometry {
  // Azimuthal mounting angle of the SSR enclosure counter-clockwise from +x (degrees).
  double ssr_angle_deg = 1;
  // Vertical offset from the kiln bottom plate to the bottom of the SSR box (mm).
  double z_ssr_offset_mm = 2;
  // Tangential width of the rear face of the SSR enclosure (mm).
  double w_ssr_mm = 3;
  // Uniform radial depth of the flat side flanges of the SSR enclosure (mm).
  double d_ssr_mm = 4;
  // Vertical height of the SSR enclosure (mm).
  double h_ssr_mm = 5;
}

// Geometry of the under-kiln reflector and multi-layer subfloor stack.
message FloorGeometry {
  // Diameter of the circular polished stainless steel / Calsil floor reflector (mm).
  double d_reflector_mm = 1;
  // Total thickness of the circular floor reflector assembly (mm).
  double t_reflector_calsil_mm = 2;
  // Vertical air gap beneath the circular floor reflector formed by spacer feet (mm).
  double g_floor_reflector_mm = 3;
  // Side width of the square non-vinyl concrete/tile topping pad in the corner (mm).
  double w_floor_pad_mm = 4;
  // Thickness of the square concrete/tile topping pad above the screed datum z = 0 (mm).
  double t_floor_pad_mm = 5;
  // Thickness of the continuous fiber-reinforced structural screed layer (mm).
  double t_screed_mm = 6;
  // Thickness of the continuous Vacuum Insulation Panel (VIP) subfloor layer (mm).
  double t_vip_mm = 7;
}

// Dimensions and placement of the overhead canopy extraction hood.
message HoodGeometry {
  // Bottom circular capture opening diameter of the canopy hood (mm).
  double d_hood_inlet_mm = 1;
  // Top exhaust duct throat diameter of the canopy hood (mm).
  double d_hood_duct_mm = 2;
  // Vertical drop distance from the ceiling (z = h_wall) down to the hood rim (mm).
  double z_hood_drop_mm = 3;
  // X-coordinate of the vertical hood centerline axis from the back wall (mm).
  double x_hood_mm = 4;
  // Y-coordinate of the vertical hood centerline axis from the left wall (mm).
  double y_hood_mm = 5;
}

// Master geometric specification combining all physical subsystems.
message Geometry {
  ShieldGeometry shield = 1;
  RoomGeometry room = 2;
```

```
  KilnGeometry kiln = 3;
  BungGeometry bung = 4;
  SsrGeometry ssr = 5;
  FloorGeometry floor = 6;
  HoodGeometry hood = 7;
}

// Operating temperatures, thermal conductances, and active airflow sources.
message HeatSourcesAndFlows {
  // Peak steady-state internal kiln firing chamber temperature (degrees Celsius).
  double t_internal_c = 1;
  // External outdoor / make-up ambient air temperature (degrees Celsius).
  double t_ext_c = 2;
  // Effective thermal transmittance of the cylindrical kiln wall (W/m^2*K).
  double u_kiln_side = 3;
  // Effective thermal transmittance of the insulated kiln lid (W/m^2*K).
  double u_kiln_top = 4;
  // Effective thermal transmittance of the insulated kiln floor slab (W/m^2*K).
  double u_kiln_bot = 5;
  // Sensible convective heat release rate exiting the open exhaust bung (W).
  double q_bung_open_w = 6;
  // Internal electrical Joule heat dissipation of the SSR heatsink (W).
  double q_ssr_internal_w = 7;
  // Active volumetric extraction flow rate of the overhead hood exhaust fan (CFM).
  double exhaust_fan_cfm = 8;
}

// Thermal emissivities (dimensionless, 0.0 to 1.0) of all solid surfaces.
message SurfaceEmissivities {
  // Emissivity of the stainless steel kiln outer jacket and lid band.
  double eps_kiln = 1;
  // Emissivity of the calcium silicate shield board (Promatect-H).
  double eps_shield = 2;
  // Emissivity of paper-faced Class A2 gypsum plasterboard.
  double eps_gypsum = 3;
  // Emissivity of bare aerated autoclaved concrete (AAC) blocks.
  double eps_aac = 4;
  // Hemispherical emissivity of the polished stainless steel reflector top face.
  double eps_reflector_top = 5;
  // Emissivity of the sheet-metal SSR electrical enclosure.
  double eps_ssr = 6;
}

// Thermal conductivities (W/m*K) and envelope U-values (W/m^2*K).
message MaterialConductivities {
  // Thermal conductivity of the calcium silicate shield and reflector backing (W/m*K).
  double k_shield = 1;
  // Thermal conductivity of standard gypsum plasterboard (W/m*K).
  double k_gypsum = 2;
  // Thermal conductivity of structural AAC blocks (W/m*K).
  double k_aac = 3;
  // Thermal conductivity of the concrete or porcelain tile floor topping pad (W/m*K).
  double k_pad = 4;
  // Thermal conductivity of the fiber-reinforced floor screed (W/m*K).
  double k_screed = 5;
  // Center-of-panel thermal conductivity of the subfloor VIP insulation (W/m*K).
  double k_vip = 6;
```

```
    // Overall thermal transmittance of the ceiling and roof assembly (W/m^2*K).
    double u_roof = 7;
    // Overall thermal transmittance of the double-glazed sliding door (W/m^2*K).
    double u_glass_door = 8;
  }

  // Master thermal properties specification.
  message ThermalProperties {
    HeatSourcesAndFlows sources = 1;
    SurfaceEmissivities emissivities = 2;
    MaterialConductivities conductivities = 3;
  }

  // Spatial discretization parameters controlling patch resolution.
  message Discretization {
    // Horizontal grid patch width along the left and back walls (mm).
    double dx_wall_mm = 1;
    // Vertical grid patch height along the left and back walls (mm).
    double dz_wall_mm = 2;
    // Number of azimuthal circumferential divisions around the kiln cylinder.
    int32 n_theta_kiln = 3;
    // Number of vertical axial divisions along the kiln cylinder height.
    int32 n_z_kiln = 4;
  }

  // Descriptive identifier for the simulation experiment.
  string experiment_name = 1;
  // Physical geometry of the kiln, shields, floor, hood, and room.
  Geometry geometry = 2;
  // Thermal boundary conditions, emissivities, and material conductivities.
  ThermalProperties thermal = 3;
  // Spatial mesh discretization settings.
  Discretization grid = 4;
}

// Top-level output record encapsulating provenance, input snapshot, derived
// geometry, safety metrics, and spatially resolved patch states.
message SimulationOutput {
  // Execution metadata and cryptographic hash linking output to input config.
  message Provenance {
    // Path to the input .textproto configuration file supplied at runtime.
    string input_config_path = 1;
    // Hexadecimal MD5 digest of the exact raw bytes of the input config file.
    string input_config_md5 = 2;
    // ISO-8601 UTC timestamp when the simulation completed.
    string timestamp_utc = 3;
    // Total wall-clock execution time of the solver in seconds.
    double solver_wall_time_s = 4;
    // True if both inner surface energy balance and middle room loop converged.
    bool converged = 5;
    // Maximum absolute energy balance residual across all patches (W/m^2).
    double max_energy_residual_w_m2 = 6;
  }

  // Geometric and thermal quantities computed deterministically from input config.
  message DerivedParameters {
    // Vertical height of the calcium silicate shield panels (mm).
```

```
  double h_shield_mm = 1;
  // Elevation of the top surface of the concrete/tile floor pad (mm).
  double z_pad_top_mm = 2;
  // Elevation of the underside of the circular floor reflector (mm).
  double z_reflector_bot_mm = 3;
  // Elevation of the top specular surface of the circular floor reflector (mm).
  double z_reflector_top_mm = 4;
  // Elevation of the underside of the kiln drum bottom plate (mm).
  double z_drum_bot_mm = 5;
  // Total vertical height of the cylindrical kiln drum plus closed lid (mm).
  double h_kiln_total_mm = 6;
  // Elevation of the top flat surface of the closed kiln lid (mm).
  double z_kiln_top_mm = 7;
  // Elevation of the bottom louver face of the SSR enclosure (mm).
  double z_ssr_bot_mm = 8;
  // Elevation of the top face of the SSR enclosure (mm).
  double z_ssr_top_mm = 9;
  // Elevation of the open top rim of the exhaust dilution cup (mm).
  double z_bung_mm = 10;
  // Elevation of the virtual mirror plane of the kiln base in the reflector (mm).
  double z_mirror_mm = 11;
  // Elevation of the lower capture rim of the overhead canopy hood (mm).
  double z_hood_inlet_mm = 12;
  // Effective hydraulic gap depth of the behind-shield chimney channel (mm).
  double d_chimney_eff_mm = 13;
  // Unshielded 1D thermal transmittance of the structural AAC wall (W/m^2*K).
  double u_aac_wall = 14;
}

// Key engineering evaluation metrics and safety pass/fail flags.
message SafetyMetrics {
  // Peak surface temperature on exposed Class A2 gypsum on the left wall (C).
  double t_gypsum_max_left_c = 1;
  // Peak surface temperature on exposed Class A2 gypsum on the back wall (C).
  double t_gypsum_max_back_c = 2;
  // True if all exposed gypsum remains strictly below 40.0 C.
  bool gypsum_safe = 3;
  // Internal air temperature surrounding the relay inside the SSR box (C).
  double t_ssr_air_c = 4;
  // Area-averaged external sheet-metal casing temperature of the SSR box (C).
  double t_ssr_case_c = 5;
  // True if t_ssr_air_c < 35.0 C and t_ssr_case_c < 45.0 C.
  bool ssr_safe = 6;
  // Temperature at the top barrier film of the VIP directly under the reflector (C).
  double t_vip_under_reflector_c = 7;
  // Peak temperature of the concrete/tile pad just outside the reflector rim (C).
  double t_floor_pad_edge_c = 8;
  // Peak temperature of the vinyl flooring just outside the square floor pad (C).
  double t_vinyl_edge_c = 9;
  // True if both under-reflector and outside-reflector floor zones are safe.
  bool floor_safe = 10;
  // Peak front-face temperature on the calcium silicate shield boards (C).
  double t_shield_max_c = 11;
  // Peak front-face temperature on the AAC block behind the shield (C).
  double t_aac_max_c = 12;
  // Maximum upward air velocity inside the behind-shield chimney gap (m/s).
  double v_chimney_max_m_s = 13;
```

```
    // Maximum exit air temperature discharging from the top of the chimney gap (C).
    double t_chimney_exit_max_c = 14;
    // True if the hood captures 100% of the primary kiln plume volume and width.
    bool plume_fully_captured = 15;
    // Fraction of primary kiln plume mass flow captured by the hood (0.0 to 1.0).
    double plume_capture_fraction = 16;
    // Fraction of behind-shield chimney exit flow captured by the hood (0.0 to 1.0).
    double chimney_capture_fraction = 17;
    // Steady-state temperature of the upper stratified room gas layer (C).
    double t_room_upper_c = 18;
    // Steady-state temperature of the lower make-up room air layer (C).
    double t_room_lower_c = 19;
    // Height of the two-zone thermal stratification interface above z = 0 (mm).
    double z_stratification_layer_mm = 20;
    // Circumferential peak-to-minimum skin temperature difference around kiln (C).
    double delta_t_kiln_skin_c = 21;
    // Temperature of the concrete/tile pad directly under the reflector center (C).
    double t_floor_pad_under_reflector_c = 22;
    // True if the VIP directly under the reflector remains strictly below 50.0 C.
    bool floor_under_reflector_safe = 23;
    // Peak temperature of the VIP outside the reflector rim under the pad edge (C).
    double t_vip_outside_reflector_c = 24;
    // True if t_vip_outside_reflector_c < 50.0 C and t_vinyl_edge_c < 45.0 C.
    bool floor_outside_reflector_safe = 25;
    // Peak surface temperature on the overhead ceiling at z = h_wall (C).
    double t_ceiling_max_c = 26;
    // True if the peak ceiling surface temperature remains strictly below 40.0 C.
    bool ceiling_safe = 27;
  }

  // Resolved thermal state of an individual discretized surface patch.
  message PatchState {
    int32 patch_id = 1;
    SurfaceType surface_type = 2;
    double x_m = 3;
    double y_m = 4;
    double z_m = 5;
    double nx = 6;
    double ny = 7;
    double nz = 8;
    double area_m2 = 9;
    double temp_front_c = 10;
    double temp_back_c = 11;
    double q_rad_net_w_m2 = 12;
    double q_conv_net_w_m2 = 13;
  }

  Provenance provenance = 1;
  SimulationConfig input_config_snapshot = 2;
  DerivedParameters derived = 3;
  SafetyMetrics metrics = 4;
  repeated PatchState patches = 5;
}
```

6. Core Internal Data Structures & Single-Patch Inspection API

6.1 `SimParams` (`src/params.py`)

- Frozen Python dataclass populated from `SimulationConfig` via `from_proto(config_pb)`.
- Converts lengths to metres, temperatures to Kelvin, flow rates to m^3/s, and angles to radians upon construction.
- Computes all derived coordinates and elevations ($z_$pad,top, $z_$reflector,bot, $z_$reflector,top, $z_$drum,bot, $z_$kiln,top, $z_$ssr
- Provides parallel envelope conductance evaluation methods `compute_ua_upper_zone(z_layer)` and `compute_ua_lower_zone(z_layer)` in W/K.
- Runs `validate_params(params)` enforcing physical feasibility: $H_$shield > 200 mm, clearances $s_$kiln $\geq$ 100 mm, chimney depths $0 \leq d_$chimney ≤ 100 mm, reflector clearances $z_$reflector,top $<$ $z_$drum,bot, pad extension under kiln footprint, and valid emissivity bounds $[0.01, 0.99]$.

6.2 `SurfaceType`, `PatchView`, `FloorSensorPoint`, and `SurfaceMesh` (`src/mesh.py`)

To combine fast vectorized NumPy matrix operations with ergonomic inspection, `SurfaceMesh` stores contiguous NumPy arrays alongside single-patch inspection methods returning self-contained `PatchView` records:

```
import dataclasses
import enum
from typing import Optional
import numpy as np


class SurfaceType(enum.IntEnum):
  """Explicitly assigned surface categories matching kiln_sim.proto."""
  UNSPECIFIED = 0
  KILN_SIDE = 1
  KILN_LID = 2
  KILN_BOTTOM = 3
  SHIELD_LEFT = 4
  SHIELD_BACK = 5
  GYPSUM_LEFT = 6
  GYPSUM_BACK = 7
  REFLECTOR_TOP = 8
  SSR_BOX = 9


@dataclasses.dataclass(frozen=True)
class PatchView:
  """Self-contained snapshot of a single discretized patch for inspection."""
  patch_id: int
  surface_type: SurfaceType
  centroid_m: tuple[float, float, float]
  normal: tuple[float, float, float]
  area_m2: float
  emissivity: float
  bounds_min_m: tuple[float, float, float]
  bounds_max_m: tuple[float, float, float]
  chimney_col_row: Optional[tuple[int, int]] = None
  temp_front_c: Optional[float] = None
  temp_back_c: Optional[float] = None
  q_rad_net_w_m2: Optional[float] = None
  q_conv_net_w_m2: Optional[float] = None
```

```python
@dataclasses.dataclass(frozen=True)
class FloorSensorPoint:
  """Point sensor on the floor for evaluating subfloor and vinyl temperatures."""
  name: str
  position_m: tuple[float, float, float]
  normal: tuple[float, float, float] = (0.0, 0.0, 1.0)
```

`SurfaceMesh` provides:

1. **`get_patch(patch_id: int, solution: Optional[SolverSolution] = None) -> PatchView`**: Returns the `PatchView` for a specific patch index, automatically attaching solved temperatures and heat fluxes if `solution` is provided.

2. **`get_patches_by_type(surface_type: SurfaceType, solution: Optional[SolverSolution] = None) -> list[PatchView]`**: Returns all patches matching a given surface type (e.g., all `SurfaceType.GYPSUM_LEFT` elements).

3. **`find_patch_at_coord(x_m: float, y_m: float, z_m: float, surface_type: Optional[SurfaceType] = None, tol_m: float = 0.025, solution: Optional[SolverSolution] = None) -> Optional[PatchView]`**: Finds the patch containing (x, y, z) within spatial tolerance `tol_m` using bounding-box checks and surface plane distance.

4. **`to_patch_protos(solution: SolverSolution) -> list[SimulationOutput.PatchState]`**: Serializes all patch geometries and solved thermal states into protobuf records.

---

7. Module Responsibilities

File	Key Classes / Functions	Purpose
`src/main.py`	`main(argv)`, `FLAGS` (`--config`, `--output_dir`, `--output_file`)	CLI entry point (`absl.app.run`). Orchestrates configuration loading, mesh generation, view factor evaluation, coupled thermal solving, safety metric extraction, and output writing.
`src/io_manager.py`	`load_simulation_config`, `write_simulation_output`, `configure_absl_logging`, `format_utc_timestamp`	Loads `.textproto` files, computes exact raw-byte MD5 digest, writes `<stem>_<YYYYMMDDTHHMMSSZ>.out.textproto`, and initializes unique per-run Abseil file logging subdirectories.
`src/params.py`	`SimParams`, `from_proto`, `validate_params`, `compute_ua_upper_zone`, `compute_ua_lower_zone`	Converts protobuf parameters to SI base units, validates geometric and physical bounds, and computes derived coordinates.
`src/mesh.py`	`SurfaceType`, `PatchView`, `FloorSensorPoint`, `SurfaceMesh`, `build_mesh`	Discretizes planar rectangular patches (vertex average centroids), curved kiln cylinder (on-skin polar midpoint centroids), disks, SSR faces, and point sensors for floor pad, vinyl, and ceiling.
`src/view_factors.py`	`compute_view_factors`, `ViewFactorSystem`, `coaxial_parallel_disks_vf`	Evaluates diffuse view factors with ray-traced cylinder occlusion, analytical coaxial disk formulas, specular reflector virtual-image matrix $F_$mirror, and sensor view factor vectors.

File	Key Classes / Functions	Purpose
`src/chimney.py`	`solve_chimney`, `ChimneySolution`, `_solve_wall_columns`	Solves 1D buoyant channel flow, friction losses, vertical air heating $T_ch(z)$, flush solid contact ($d_chimney = 0$), stagnant cavity limits, and 1D AAC wall conduction.
`src/plume_room.py`	`solve_plume_and_room`, `RoomSolution`	Evaluates corner-modified MTT buoyant plume (m_eff), canopy hood capture fraction (η_plume), doorway buoyancy exchange, and two-zone room air temperatures ($T_room,upper, T_room,lower$).
`src/solver.py`	`solve_thermal_system`, `SolverSolution`, `solve_radiosity_system`	Vectorized diagonal-Newton solver with exact analytical derivatives dR_m/dT_m, pre-factored LU radiosity solve, and coupled outer room convergence loop.
`src/metrics.py`	`compute_safety_metrics`, `build_simulation_output`, `_solve_floor_sensor_temp_k`	Solves point sensor thermal equilibria (pad edge, outside VIP, vinyl edge, ceiling), evaluates pass/fail thresholds, and populates `SimulationOutput`.
`run_sweep.py`	`SweepPoint`, `enumerate_sweep_points`, `_run_geometry_group`, `reservoir_sample_points`	Parallel parametric sweep driver with `ProcessPoolExecutor`, mesh and view factor caching across runs sharing `geometry_cache_key`, reservoir sampling (`--quick`), and CSV generation (`--csv_output`).
`tests/`	Unit and integration test suite	Comprehensive test coverage across all modules (`params_test.py`, `mesh_test.py`, `view_factors_test.py`, `chimney_test.py`, `solver_test.py`, `run_sweep_test.py`).

---

8. Bazel Build & Execution Commands

1. Compile and Run the Unit Test Suite

```
# Run all unit and integration tests
bazel test //tests:all

# Run specific individual test targets
bazel test //tests:params_test
bazel test //tests:mesh_test
bazel test //tests:view_factors_test
bazel test //tests:chimney_test
bazel test //tests:solver_test
bazel test //tests:run_sweep_test
```

2. Execute a Single Simulation Run

```
# Run single simulation (writes timestamped .out.textproto into results/)
bazel run //src:main -- \
    --config="$(pwd)/configs/baseline_205mm.textproto" \
    --output_dir="$(pwd)/results"
```

```
# Verify input config cryptographic provenance from output record
md5sum configs/baseline_205mm.textproto
grep "input_config_md5" results/baseline_205mm_*.out.textproto
```

3. Execute Parametric Sweeps (`run_sweep.py`)

```
# Fast smoketest: sample 2 * max_workers configurations via reservoir sampling
bazel run //:run_sweep -- --quick

# Generate sweep configuration .textproto files without running solver
bazel run //:run_sweep -- --configs_only

# Full parametric sweep across all combinations using 8 parallel worker processes
bazel run //:run_sweep -- --jobs=8

# Aggregate existing .out.textproto files into summary CSV without re-running
bazel run //:run_sweep -- --csv_only --csv_output="$(pwd)/sweep_results.csv"
```